Tic-Tac-Toe Online Game - Documentation

Student Information

Name	ID
Malak mrowat	212681282
Jobran khateeb	207719600
Besan ali	214295552

Field	Value
Operating System	Windows 11
Python Version	Python 3.11

Project Overview

This project implements a multi-player online Tic-Tac-Toe game using Python sockets with a client-server architecture. The game supports 2-10 players with dynamic board sizes.

Game Rules

- **x players** use a board of size **$(x+1)^2$**
  - 2 players: 3x3 board
  - 3 players: 4x4 board
- **3 in a row** to win (horizontal, vertical, or diagonal)
- Player symbols: X, O, Δ, □, ◇, etc.

Files Description

1. server.py

The game server that manages all game sessions and client connections.

Classes

Game - Represents a single Tic-Tac-Toe game session.

Method	Description
__init__(game_id, num_players)	Initializes game with ID and number of players. Creates board of size $(num_players+1)^2$
add_player(client_socket)	Adds a player to the game and assigns them a symbol
is_full()	Returns True if all players have joined
start_game()	Marks the game as started
get_current_symbol()	Returns the symbol of the player whose turn it is
make_move(row, col, symbol)	Attempts to place a symbol at the given position. Returns success status and result
check_winner(symbol)	Checks if the given symbol has 3 in a row
check_draw()	Checks if the board is full (draw)
get_board_string()	Serializes the board for network transmission
remove_player(client_socket)	Removes a player from the game
broadcast(message, exclude)	Sends a message to all players in the game

Functions

Function	Description
handle_client(client_socket, client_address)	Handles communication with a single client. Processes commands: CREATE, LIST, JOIN, MOVE, QUIT
admin_console(server_socket)	Provides server admin interface. Type 'stop' to shut down
main()	Entry point. Creates TCP socket, starts admin console thread, and accepts client connections

Global Variables

Variable	Description
HOST	Server IP address (127.0.0.1)
PORT	Server port (5555)

Variable	Description
SYMBOLS	List of player symbols: X, O, Δ, □, ◇, etc.
games	Dictionary storing all active games
games_lock	Threading lock for thread-safe game access

---

2. client.py

The game client that connects to the server and provides the user interface.

Functions

Function	Description
print_board(board)	Displays the game board with row/column numbers
parse_board(board_str)	Parses the board string from server into a 2D list
receive_messages(sock)	Thread function that receives and processes server messages
print_menu()	Displays the main menu with available commands
main()	Entry point. Connects to server, starts receiver thread, handles user input

Global Variables

Variable	Description
HOST, PORT	Server connection details
my_symbol	The player's assigned symbol
board_size	Current game board size
current_board	Current state of the game board
current_turn	Symbol of the player whose turn it is
in_game	True if currently in a game
game_over	True if the game has ended
running	True while the client is running

Communication Protocol

Client to Server Commands

Command	Description
CREATE <n>	Create a new game for n players
LIST	Get list of available games
JOIN <id>	Join game with specified ID
MOVE <row> <col>	Make a move at position (row, col)
QUIT	Disconnect from server

Server to Client Responses

Response	Description
CREATED <id>	Game created successfully
GAMES <list>	List of available games
JOINED <id>	Successfully joined game
WAITING <n>/<total>	Waiting for more players
START <symbol> <size>	Game started, assigned symbol and board size
BOARD <state>	Current board state
TURN <symbol>	Indicates whose turn it is
WIN <symbol>	Game over, winner announced
DRAW	Game ended in a draw
ERROR <msg>	Error message
PLAYER_LEFT <symbol>	A player disconnected

Usage Examples

1. Starting the Server

```
PS C:\Users\Administrator\Desktop\aaaa> python server.py
[SERVER STARTED] Listening on 127.0.0.1:5555
[ADMIN] Type 'stop' to shut down the server
```

2. Starting the Client

```
PS C:\Users\Administrator\Desktop\aaaa> python client.py
[CONNECTED] Connected to server at 127.0.0.1:5555

======================================
       TIC-TAC-TOE GAME
======================================
Commands:
  CREATE <num_players> - Create a new game
  LIST                 - List available games
  JOIN <game_id>       - Join an existing game
  QUIT                 - Exit

In game:
  <row> <col>          - Make a move (e.g., 0 1)
======================================
```

3. Player Creating a Game

```
PS C:\Users\Administrator\Desktop\aaaa> python client.py
[CONNECTED] Connected to server at 127.0.0.1:5555

======================================
       TIC-TAC-TOE GAME
======================================
Commands:
  CREATE <num_players> - Create a new game
  LIST                 - List available games
  JOIN <game_id>       - Join an existing game
  QUIT                 - Exit

In game:
  <row> <col>          - Make a move (e.g., 0 1)
======================================
create 3

[GAME] Created game #1. Waiting for players...
[GAME] Players: 1/3
```

4. Another Player Listing and Joining

```
PS C:\Users\Administrator\Desktop\aaaa> python client.py
[CONNECTED] Connected to server at 127.0.0.1:5555

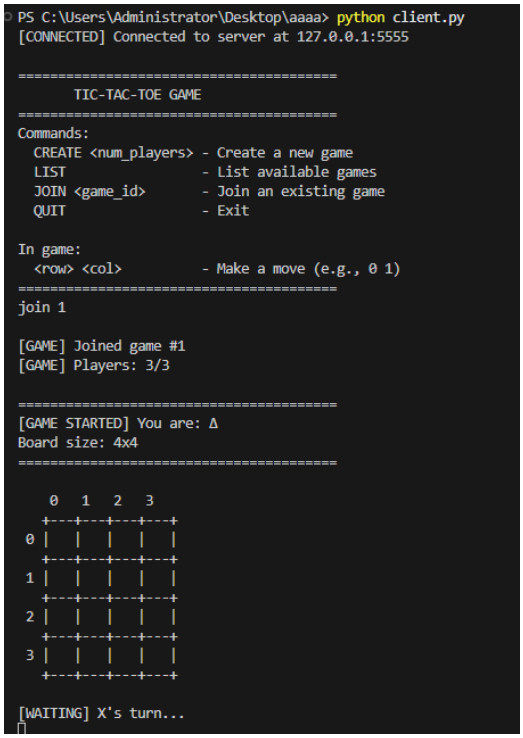
======================================
       TIC-TAC-TOE GAME
======================================
Commands:
  CREATE <num_players> - Create a new game
  LIST                 - List available games
  JOIN <game_id>       - Join an existing game
  QUIT                 - Exit

In game:
  <row> <col>          - Make a move (e.g., 0 1)
======================================
list

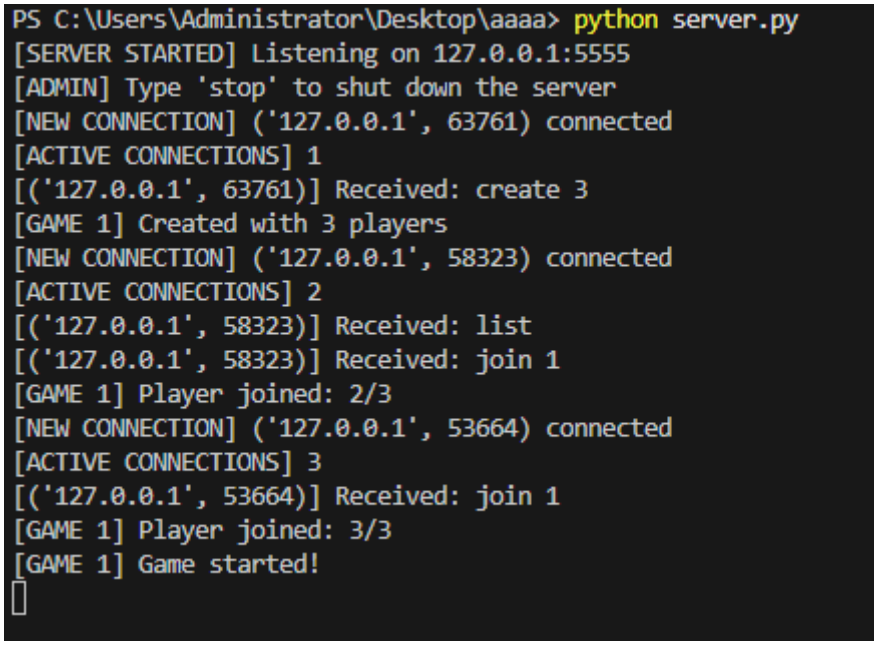
[GAMES] Available games:
  Game #1: 1/3 players
join 1

[GAME] Joined game #1
[GAME] Players: 2/3
```

5. Third Player Joins



6. Server Logs



7. Player Wins



How to Run

Step 1: Start the Server

python server.py

Step 2: Start Client(s)

python client.py

Step 3: Create or Join a Game

- Type CREATE 2 to create a 2-player game
- Type LIST to see available games
- Type JOIN 1 to join game #1

Step 4: Play

- When it's your turn, type coordinates: 0 1 (row 0, column 1)
- The game ends when someone gets 3 in a row or the board is full

Step 5: Stop the Server

- Type stop in the server console

Technical Notes

- Uses TCP sockets for reliable communication
- Multi-threading handles multiple concurrent clients and games
- UTF-8 encoding for Unicode symbol support (Δ)
- Thread-safe game management using locks